

概要设计说明书

项目名称：在线论文文档自动生成工具

版本：V2026.07

成员：	<u>42411051 李思贤</u>
成员：	<u>42411084 余锦标</u>
成员：	<u>42411094 谢宏涛</u>
成员：	<u>42411116 李佳蔚</u>
成员：	<u>42411134 张宗元</u>
成员：	<u>42411202 张熙伟</u>

实训课程软件工程文档

2026 年 7 月 16 日

目录

目录	1
1 系统总体架构	6
1.1 架构设计原则	6
1.2 B/S 三层架构	6
1.3 前后端分离交互流程	7
1.4 系统模块划分	7
2 技术选型	7
2.1 技术栈总览	7
2.2 后端技术选型分析	9
2.2.1 Spring Boot 4.1.0 + Java 21	9
2.2.2 Spring Data JPA + PostgreSQL 16	9
2.2.3 Redis 7	10
2.3 认证与安全方案	10
2.4 前端技术选型分析	11
2.4.1 Vue 3 + Vite	11
2.4.2 Axios + Vue Router	11
2.5 文档生成方案	11
3 数据库设计	11
3.1 数据库概述	11
3.2 实体关系设计	11
3.3 数据库表清单	12
3.4 核心表结构设计	12
3.4.1 用户表 (sys_user)	12
3.4.2 论文表 (thesis)	12
3.4.3 模板版本表 (template_version)	14
3.5 索引设计	14
4 开发环境与工程结构	15
4.1 开发环境配置	15
4.2 后端工程结构	15
5 模板管理模块概要设计	16
5.1 模块定位	16
5.2 模块边界与依赖关系	16
5.3 模块内部结构	17
5.4 核心实体 E-R 关系	17
6 学院模块接口设计	17

7 模板模块接口设计	17
7.1 模板版本 JSON 配置数据结构	17
8 论文模块概要设计	19
8.1 模块定位	19
8.2 论文状态机设计	19
8.3 论文锁定机制	19
9 论文模块接口设计	20
10 章节存取模块概要设计	20
10.1 章节树存储模型	20
11 章节存取接口设计	21
12 数据库表结构汇总	21
13 参考文献管理模块概要设计	21
13.1 模块职责	21
13.2 模块接口	21
13.3 与外部模块的交互	21
13.4 GB/T 7714 格式化设计	23
14 图片上传与管理模块概要设计	23
14.1 模块职责	23
14.2 模块接口	23
14.3 技术方案	23
14.4 与外部模块的交互	24
15 文档导出模块概要设计	24
15.1 模块职责	24
15.2 总体流程	24
15.3 技术方案对比	24
15.4 数据流设计	25
15.5 与外部模块的交互	25
16 前端技术选型与架构概述	26
16.1 技术栈	26
16.2 选型理由	26
17 前端工程目录结构	26
18 前端路由设计	28
18.1 路由表定义	28
18.2 路由鉴权守卫	28

19 前端组件架构设计	29
19.1 组件分层架构	29
19.2 核心组件通信模式	29
19.3 编辑器组件架构	30
19.4 组件状态生命周期	30
20 Axios 封装与 API 层设计	31
20.1 请求/响应拦截器	31
20.2 学生端 API 接口清单	33
21 学生端核心模块概要设计	33
21.1 论文创建向导流程	33
21.2 编辑器核心逻辑	33
21.3 论文预览	33
22 管理员端页面概要设计	34
22.1 页面布局	34
22.2 仪表盘页设计	34
22.3 学院管理页设计	34
22.4 模板管理页设计	34
22.5 模板版本编辑页设计	35
23 教师端页面概要设计	35
23.1 页面布局	35
23.2 批阅任务列表页设计	35
23.3 论文批阅页设计	35
23.4 评分对话框设计	36
24 UI 设计规范	36
24.1 颜色系统	36
24.2 字号层级	36
24.3 间距规范	37
24.4 响应式断点	37
24.5 组件使用约定	37
25 缓存与部署架构设计	37
25.1 缓存架构概述	38
25.1.1 缓存架构分层	38
25.1.2 缓存数据流	38
25.2 Redis 数据结构设计	38
25.2.1 序列化方案	39
25.2.2 连接池配置	39
25.3 缓存策略详解	39
25.3.1 缓存预热	39

25.3.2	缓存失效策略	40
25.3.3	缓存穿透防护	40
25.3.4	缓存雪崩防护	40
25.3.5	缓存击穿防护	40
25.4	部署架构概述	40
25.4.1	容器拓扑	42
25.4.2	各组件说明	42
25.4.3	数据卷规划	42
25.4.4	网络规划	42
25.4.5	健康检查与重启策略	42
25.4.6	环境配置分层	42
25.4.7	资源限制	42
25.4.8	部署流程	43

版本修订记录

版本	日期	修改人	审核	修改说明
V1.0	2026-07-16	全组	全组	初稿完成

1 系统总体架构

1.1 架构设计原则

本系统遵循以下架构设计原则：

- 1. **前后端分离**：前端采用 Vue 3 单页应用（SPA），后端采用 Spring Boot RESTful API，前后端通过 HTTP/JSON 协议通信，实现彻底的前后端解耦，便于独立开发、测试和部署；
- 2. **分层架构**：后端采用 Controller-Service-Repository 三层架构，各层职责明确、单向依赖，上层调用下层接口，下层不感知上层逻辑；
- 3. **无状态服务**：API 服务不持有会话状态，认证信息通过 JWT Token 在客户端和服务端之间传递，Token 缓存至 Redis 以保证分布式环境下的一致性；
- 4. **数据一致性**：核心业务数据采用关系型数据库 PostgreSQL 存储，利用外键约束、事务管理和数据库索引保证数据完整性和查询性能；
- 5. **缓存策略**：高频读取的热点数据（模板配置、用户会话）缓存在 Redis 中，减少数据库查询压力，提升系统响应速度。

1.2 B/S 三层架构

系统采用经典的 B/S（Browser/Server）三层架构模型，从逻辑上划分为表现层、业务逻辑层和数据访问层，如图 1 所示。



图 1: B/S 三层架构示意图

各层职责说明如下：

- **表现层**：负责用户界面展示和交互，基于 Vue 3 Composition API 构建组件化页面，使用 Element Plus 组件库统一 UI 风格，通过 Axios 封装 HTTP 请求与后端 API 通信。页面包括登录注册、论文编辑、模板管理、批阅审核等功能模块；

- **业务逻辑层**：作为系统核心，基于 Spring Boot 框架实现全部业务逻辑。Controller 层负责接收 HTTP 请求、参数校验和响应格式化；Service 层封装业务规则（用户认证、论文管理、模板控制、文档导出、批注批阅等）；Repository 层通过 Spring Data JPA 完成数据持久化操作；
- **数据访问层**：PostgreSQL 存储全部结构化业务数据（用户、论文、模板、批注等），Redis 提供 Token 缓存和热点数据缓存服务，服务器本地文件系统存储上传的图片资源和生成的 DOCX/PDF 文档文件。

1.3 前后端分离交互流程

前后端分离架构下的典型请求处理流程如下：

1. 用户在浏览器中操作，Vue 3 前端通过 Axios 发起 HTTP 请求（GET/POST/PUT/DELETE），请求头携带 JWT Token（格式：Authorization: Bearer <token>）；
2. 请求到达 Spring Boot 内嵌 Tomcat 服务器，首先经过 JWT 过滤器（JwtFilter）解析 Token 并将用户信息（userId、username、role）注入请求上下文；
3. 请求分发至对应的 Controller 方法，Spring MVC 完成参数绑定和 JSR-303 校验（@Valid）；
4. Controller 调用 Service 层执行业务逻辑，Service 通过 JPA Repository 与 PostgreSQL 数据库交互，必要时读写 Redis 缓存；
5. Service 将处理结果返回 Controller，Controller 通过统一返回体 Result 封装响应数据（code、message、data）返回前端；
6. 前端根据响应状态码和数据进行页面渲染或错误提示。

1.4 系统模块划分

系统按功能划分为六大核心模块，如表 1 所示：

六大模块的依赖关系为：M1 认证鉴权模块为所有模块提供统一的身份认证和权限校验基础；M2 模板论文模块是核心业务模块，M3 参考文献模块和 M4 导出模块依赖于 M2 的论文数据；M5 前端模块为 M1-M4 提供用户操作界面；M6 提交批阅模块串联学生、教师角色的协同工作流。

2 技术选型

2.1 技术栈总览

系统技术栈如表 2 所示，每一项技术的选择均基于明确的技术理由和项目约束。

表 1: 系统功能模块划分

模块编号	模块名称	功能说明
M1	认证与权限管理模块	用户注册、登录、登出, JWT Token 签发与验证, 角色权限控制 (ADMIN/STUDENT/TEACHER), 密码 BCrypt 加密存储
M2	模板与论文管理模块	管理员按学院管理论文模板 (封面字段、章节结构、格式参数), 支持模板版本管理; 学生创建论文、在线编辑章节内容 (富文本)、草稿自动保存与手动保存
M3	参考文献与图片管理模块	学生逐条录入参考文献信息 (支持 BibTeX 批量导入), 上传论文插图 (格式/大小校验), 图片内联展示
M4	文档导出模块	基于论文内容 + 模板格式配置, 服务端异步生成标准 DOCX (Apache POI) 和 PDF (LaTeX 编译) 文档, 提供下载
M5	前端交互与 UI 模块	Vue 3 组件化页面构建, 包括学生端 (论文编辑、预览、导出、提交)、教师端 (批注、评阅、退回)、管理员端 (模板管理、学院管理)
M6	提交批阅与缓存部署模块	学生论文提交/教师批阅/退回重提完整流程, Redis 缓存策略, 系统一键部署配置 (Docker Compose)

表 2: 系统技术栈总览

层次	技术	版本
后端框架	Spring Boot	4.1.0
编程语言	Java	21 (LTS)
Web 框架	Spring Web MVC (spring-boot-starter-webmvc)	4.1.0
ORM 框架	Spring Data JPA (Hibernate)	4.1.0
关系型数据库	PostgreSQL	16+
缓存中间件	Redis	7+
认证	JWT (io.jsonwebtoken)	0.12.6
密码加密	Spring Security Crypto (BCrypt)	6.x
API 文档	Knife4j (Swagger/OpenAPI 3)	4.5.0
前端框架	Vue 3 (Composition API)	3.x
构建工具	Vite	5.x
UI 组件库	Element Plus	2.x
HTTP 客户端	Axios	1.x
前端路由	Vue Router	4.x
DOCX 生成	Apache POI (poi-ooxml)	5.x
PDF 生成	LaTeX (xelatex) + 模板类	TeX Live 2024+
构建管理	Maven (mvnw wrapper)	3.9+

2.2 后端技术选型分析

2.2.1 Spring Boot 4.1.0 + Java 21

Spring Boot 4.1.0 是 Spring 生态最新的稳定主版本，基于 Java 21 LTS（长期支持版本）构建。选择理由如下：

- **约定优于配置：**自动配置机制大幅减少 XML 配置，通过 application.properties 管理全部参数，开发效率高；
- **内嵌 Tomcat：**无需单独部署 Web 容器，应用打包为可执行 JAR，配合 Maven Wrapper 实现零环境依赖启动；
- **生态成熟：**与 Spring Data JPA、Spring Web MVC、Spring Security Crypto 等子项目无缝集成；
- **Java 21 特性：**支持虚拟线程（Virtual Threads）、记录类（Records）、模式匹配（Pattern Matching）等现代语言特性，提升代码可读性和并发处理能力；
- **生产就绪：**内置 Actuator 健康检查、Metrics 监控端点，便于运维巡检。

2.2.2 Spring Data JPA + PostgreSQL 16

数据持久化层采用 JPA（Java Persistence API）规范实现，选择理由：

- **对象关系映射**: Java 实体类通过注解直接映射数据库表结构, 减少手写 SQL, 提高开发效率;
- **方法命名查询**: Spring Data JPA 通过 Repository 接口的方法命名规则自动生成查询 SQL, 常见 CRUD 操作无需编写 JPQL;
- **数据库迁移友好**: 通过 `spring.jpa.hibernate.ddl-auto=validate` 在校验模式下确保实体定义与数据库结构一致;
- **PostgreSQL 优势**: 支持 JSONB 类型 (模板配置的灵活存储)、丰富的索引类型 (B-tree、GIN)、ACID 事务保证、成熟的并发控制 (MVCC);
- **外键约束**: 利用数据库级外键保证引用完整性, 防止数据孤岛。

2.2.3 Redis 7

Redis 作为缓存层和 Token 存储, 选择理由:

- **高性能**: 纯内存操作, 读写延迟在毫秒级, 单机 QPS 可达 10 万 +;
- **数据结构丰富**: String 类型存储 Token 和序列化对象 (JSON), 支持设置过期时间 (TTL), 天然适合 JWT Token 的 24 小时过期管理;
- **轻量级**: 部署和维护成本低, 实训环境中单实例即可满足需求。

2.3 认证与安全方案

系统未引入 Spring Security 全套框架, 而是采用轻量级的自定义认证方案:

- **JWT Token (jjwt 0.12.6)**: 无状态 Token 认证, 服务端不保存会话, Token 自带用户 ID、用户名、角色信息, 通过 HMAC-SHA256 签名防篡改, 有效期 24 小时;
- **BCrypt 密码加密 (spring-security-crypto)**: 单向哈希加盐算法, 相同密码每次加密结果不同, 有效防御彩虹表攻击, work factor 默认 10 轮;
- **角色级权限控制**: 自定义 `@RoleRequired` 注解配合 Spring MVC 拦截器, 在 Controller 方法级别声明所需的角色 (ADMIN/STUDENT/TEACHER), 拦截器在请求进入 Controller 前校验当前用户角色是否在允许列表中;
- **前后端双重校验**: 前端路由守卫隐藏无权限页面入口, 后端 API 强制校验 JWT Token 和角色权限, 防止绕过前端直接调用 API。

2.4 前端技术选型分析

2.4.1 Vue 3 + Vite

Vue 3 采用 Composition API 编程范式,相比 Options API 具有更好的逻辑复用性和 TypeScript 支持; Vite 作为下一代前端构建工具,利用 ES 模块原生支持实现极速冷启动和热更新 (HMR),开发体验显著优于 Webpack。Element Plus 是基于 Vue 3 的成熟 UI 组件库,提供表单、表格、对话框、上传、富文本等丰富的企业级组件,降低前端开发工作量。

2.4.2 Axios + Vue Router

Axios 封装 HTTP 请求,统一配置 Base URL、请求超时、拦截器 (请求前自动注入 JWT Token,响应时统一处理 401 未授权跳转); Vue Router 4 实现前端单页路由,通过嵌套路由组织页面层级 (Layout 布局页 → 功能子页面),路由守卫在页面切换前校验用户登录状态和角色权限。

2.5 文档生成方案

文档导出是本系统的核心功能之一,采用两种技术路径:

- **DOCX 生成 (Apache POI):** Apache POI 是 Java 领域操作 Microsoft Office 文档的事实标准库,通过 poi-ooxml 模块可编程创建 DOCX 文件。服务端根据论文 HTML 内容解析文本段落、样式标记、图片引用,结合模板格式配置 (字体、字号、行距、页边距),按序构建 XWPFDocument 对象并输出 DOCX 字节流;
- **PDF 生成 (LaTeX + xelatex):** 利用已有的 LaTeX 模板类 (softeng.cls) 和编译流程,服务端将论文内容转换为 LaTeX 源码文件,调用 xelatex 编译器生成高质量 PDF 文档。LaTeX 排版引擎在数学公式、参考文献格式化、目录自动生成方面具有不可替代的优势。

3 数据库设计

3.1 数据库概述

系统数据库名称为 `thesis_generator`,使用 PostgreSQL 16 关系型数据库。数据库设计遵循第三范式 (3NF),核心实体间通过外键建立关联约束,保证数据引用完整性。共设计 10 张核心数据表和 1 张辅助历史记录表。

3.2 实体关系设计

系统的核心实体及其关系如下:

- **学院 (College) 与用户 (User):** 一对多关系。一个学院包含多名用户 (学生和教师),用户通过 `college_id` 外键关联所属学院;

- **学院 (College) 与模板 (Template):** 一对多关系。一个学院下可有多个论文模板, 模板通过 college_id 外键关联所属学院;
- **模板 (Template) 与模板版本 (TemplateVersion):** 一对多关系。一个模板可经历多次修订, 每次修订记录为一个模板版本, 模板版本通过 template_id 外键关联所属模板;
- **模板版本 (TemplateVersion) 与论文 (Thesis):** 一对多关系。一个模板版本可被多篇论文引用, 论文通过 template_version_id 外键关联模板版本, 同时记录所属学院;
- **用户 (User) 与论文 (Thesis):** 一对多关系。一名学生可创建多篇论文, 论文通过 student_id 外键关联学生用户;
- **论文 (Thesis) 与章节 (ThesisSection):** 一对多关系。一篇论文包含多个章节 (引言、各论文章节、结论、参考文献等), 章节通过 thesis_id 外键关联所属论文, 通过 parent_id 自引用实现章节嵌套;
- **论文 (Thesis) 与参考文献 (ThesisReference):** 一对多关系。一篇论文包含多条参考文献条目;
- **论文 (Thesis) 与图片 (ThesisImage):** 一对多关系。一篇论文中可嵌入多张图片;
- **论文 (Thesis) 与提交记录 (Submission):** 一对多关系。一篇论文可有多次提交记录 (学生修改后重新提交);
- **论文 (Thesis) 与批注 (Annotation):** 一对多关系。一篇论文可有教师添加的多条批注, 批注关联到具体章节和教师;
- **论文 (Thesis) 与批阅记录 (ReviewRecord):** 一对多关系。一篇论文可被多次批阅或退回。

3.3 数据库表清单

系统全部数据表及其职责如表 3 所示:

3.4 核心表结构设计

3.4.1 用户表 (sys_user)

用户表是系统权限体系的基础, 结构如表 4 所示:

3.4.2 论文表 (thesis)

论文表是系统核心业务数据表, 结构如表 5 所示:

论文状态流转规则为: DRAFT (编辑中) → COMPLETED (完成) → SUBMITTED (已提交) → REVIEWED (已批阅) / RETURNED (已退回, 回到 DRAFT)。

表 3: 数据库表清单

序号	表名	说明
1	sys_college	学院信息表, 存储学院名称和编号
2	sys_user	系统用户表, 存储全部用户账号、角色和所属学院
3	template	论文模板表, 存储模板基本信息和启用状态
4	template_version	模板版本表, 存储每个模板各版本的配置数据 (JSONB)
5	thesis	论文主表, 存储每篇论文的元信息
6	thesis_section	论文章节表, 存储章节内容和结构信息
7	thesis_reference	参考文献表, 存储论文的参考文献条目
8	thesis_image	图片资源表, 记录上传图片的文件信息
9	submission	提交记录表, 记录学生历次提交信息
10	annotation	批注表, 存储教师在论文指定位置的批注意见
11	review_record	批阅记录表, 存储教师的评语、分数和操作类型
12	thesis_section_draft_history	章节草稿历史表, 支持自动保存版本回退

表 4: sys_user 用户表结构

字段名	数据类型	约束	说明
id	BIGSERIAL	PK	自增主键
username	VARCHAR(50)	NOT NULL, UNIQUE	登录用户名
password	VARCHAR(255)	NOT NULL	BCrypt 加密密码
real_name	VARCHAR(50)	NOT NULL	用户真实姓名
role	VARCHAR(20)	NOT NULL, CHECK	ADMIN / STUDENT / TEACHER
college_id	BIGINT	FK → sys_college.id	所属学院
enabled	BOOLEAN	DEFAULT TRUE	账号启用状态
created_at	TIMESTAMP	DEFAULT NOW	创建时间
updated_at	TIMESTAMP	DEFAULT NOW	更新时间

表 5: thesis 论文表结构

字段名	数据类型	约束	说明
id	BIGSERIAL	PK	自增主
student_id	BIGINT	NOT NULL, FK	学生用户
template_version_id	BIGINT	FK	关联模板
title	VARCHAR(500)	NOT NULL	论文标
status	VARCHAR(20)	NOT NULL, CHECK	DRAFT/COMPLETED/SUBMITTED/REV
is_locked	BOOLEAN	DEFAULT FALSE	提交后锁
college_id	BIGINT	FK	所属学
created_at	TIMESTAMP	DEFAULT NOW	创建时
updated_at	TIMESTAMP	DEFAULT NOW	更新时

3.4.3 模板版本表 (template_version)

模板版本表利用 PostgreSQL JSONB 类型实现灵活的配置存储，结构如表 6 所示：

表 6: template_version 模板版本表结构

字段名	数据类型	约束	说明
id	BIGSERIAL	PK	自增主键
template_id	BIGINT	NOT NULL, FK	所属模板
version_number	VARCHAR(20)	NOT NULL	版本号（如 1.0、2.0）
is_current	BOOLEAN	DEFAULT TRUE	是否为当前版本
cover_fields	JSONB	NOT NULL	封面字段配置（JSON 数组）
chapter_structure	JSONB	NOT NULL	章节结构定义（JSON 数组）
format_config	JSONB	NOT NULL	排版格式参数（JSON 对象）
created_at	TIMESTAMP	DEFAULT NOW	创建时间

其中cover_fields 示例:["title", "author", "studentId", "college", "major", "advisor", "date"]
chapter_structure 示例指定章节层级和默认标题；format_config 包含字体、字号、行距、页边距等排版参数。

3.5 索引设计

为保障查询性能，系统在以下字段上建立了索引，如表 7 所示：

索引策略遵循以下原则：高频查询条件字段建索引（如 role、status），外键关联字段建索引（如 student_id、thesis_id、college_id），避免在频繁更新的字段（如 content）上建立索引。当前索引方案可覆盖系统的全部核心查询场景。

表 7: 数据库索引设计

索引名	表	字段	用途
idx_user_role	sys_user	role	按角色筛选用户
idx_template_college	template	college_id	按学院查询模板
idx_template_type	template	type	按类型筛选模板
idx_thesis_student	thesis	student_id	查询学生论文列表
idx_thesis_status	thesis	status	按状态筛选论文
idx_section_thesis	thesis_section	thesis_id	查询论文全部章节
idx_annotation_thesis	annotation	thesis_id	查询论文全部批注
idx_review_thesis	review_record	thesis_id	查询论文批阅记录

4 开发环境与工程结构

4.1 开发环境配置

表 8: 开发环境要求

工具/环境	版本要求	用途
JDK	21 LTS	Java 编译和运行
Maven	3.9+（已内置 mvnw）	项目构建与依赖管理
PostgreSQL	16+	关系型数据库
Redis	7+	缓存与 Token 存储
Node.js	18+	前端构建和开发服务器
TeX Live	2024+	LaTeX 文档编译（PDF 导出）
Git	2.40+	版本控制
IDE	IntelliJ IDEA / VS Code	代码开发

4.2 后端工程结构

后端工程遵循 Maven 标准目录结构，采用分包分层组织代码：

Listing 1: 后端工程目录结构

```
thesis-generator/  
+-- pom.xml                # Maven 配置（依赖/插件）  
+-- mvnw / mvnw.cmd        # Maven Wrapper（免安装）  
+-- src/main/java/com/example/thesisgenerator/  
|   +-- ThesisGeneratorApplication.java # Spring Boot 启动类  
|   +-- common/                # 通用组件  
|   |   +-- Result.java        # 统一 API 返回体  
|   |   +-- BusinessException.java # 业务异常类  
|   |   +-- GlobalExceptionHandler.java # 全局异常处理  
|   +-- config/                # 配置类
```



```

11 | | +-- CorsConfig.java           # CORS跨域配置
12 | | +-- RedisConfig.java        # Redis序列化配置
13 | | +-- JwtUtil.java           # JWT工具类
14 | | +-- JwtFilter.java         # JWT认证过滤器
15 | | +-- RoleRequired.java      # 角色权限注解
16 | | +-- RoleInterceptor.java   # 角色校验拦截器
17 | | +-- WebConfig.java         # MVC拦截器注册
18 | | +-- SwaggerConfig.java     # API文档配置
19 | +-- entity/                  # JPA实体类
20 | +-- repository/              # 数据访问层
21 | +-- dto/                     # 数据传输对象
22 | +-- service/                 # 业务逻辑层
23 | +-- controller/             # RESTful控制器
24 +-- src/main/resources/
25 | +-- application.properties   # 应用配置
26 | +-- schema.sql               # 数据库初始化脚本
27 +-- src/test/                  # 单元测试

```

该结构的优势为：代码按职责分层（controller/service/repository），再按业务模块分包，层次清晰、职责分明，新成员可快速定位代码位置。

5 模板管理模块概要设计

5.1 模块定位

模板管理模块是论文自动生成系统的排版基础层，位于系统三层架构中的业务逻辑层。该模块负责定义和管理论文的全部排版规范参数，包括封面样式、章节结构骨架、字体字号行距、页边距页眉页脚等。上层模块——论文撰写编辑模块（D 负责）和文档导出生成模块（C 负责）——均依赖模板模块提供的 JSON 配置数据进行内容渲染和格式化输出。

5.2 模块边界与依赖关系

模板管理模块的边界清晰，仅与以下模块存在依赖交互：

- **依赖认证权限模块（A 负责）**：模板管理的所有 API 接口均需通过 JWT Token 鉴权，仅系统管理员角色可访问。
- **被论文撰写模块（D 负责）调用**：学生创建论文时查询可用模板列表、选择模板版本；编辑器加载时获取模板的 chapterStructure 构建章节树。
- **被文档导出模块（C 负责）调用**：导出 DOCX/PDF 时读取模板的 coverFields 渲染封面、formatConfig 设置文档样式。

5.3 模块内部结构

模板管理模块内部划分为三个子模块：

1. **学院管理子模块**：提供学院信息的 CRUD 接口，为模板按学院维度分类提供基础数据支撑。
2. **模板管理子模块**：管理模板元数据（名称、类型、关联学院），每个模板维护一个独立的版本链。
3. **版本管理子模块**：管理每个模板的版本记录，存储完整的排版配置数据（三个 JSON 字段：coverFields、chapterStructure、formatConfig）。

5.4 核心实体 E-R 关系

- **College → Template**：一对多关系。一个学院可有多个专属模板，同时可存在通用模板（college_id 为 NULL）。
- **Template → TemplateVersion**：一对多关系。一个模板可有多个历史版本，其中有且仅有一个版本 is_current 为 true。
- **TemplateVersion → Thesis**：一对多关系。学生创建论文时选择一个模板版本作为快照，论文生成时使用该快照的配置，不受后续模板版本变更影响。

6 学院模块接口设计

表 9: 学院管理 RESTful API

方法	URL	权限	说明
GET	/api/colleges	Admin	获取全部学院列表（按 name 升序），返回 id、name、code、创建时间
POST	/api/colleges	Admin	新增学院。请求体：{ "name": "...", "code": "..." }, code 唯一性校验
PUT	/api/colleges/{id}	Admin	编辑学院信息（仅 name 可修改，code 创建后不可改）
DELETE	/api/colleges/{id}	Admin	删除学院。删除前检查关联数据，有关联则返回错误

7 模板模块接口设计

7.1 模板版本 JSON 配置数据结构

TemplateVersion 实体通过三个 JSON 格式的 TEXT 字段存储模板的完整配置参数。

表 10: 模板管理 RESTful API

方法	URL	权限	说明
GET	/api/templates	Admin	获取模板列表。支持查询参数 type、collegeId 筛选
POST	/api/templates	Admin	创建模板。请求体: { "name": "...", "type": "GRADUATION", "collegeId": 1 }。同时创建 V1.0 版本
PUT	/api/templates/{id}	Admin	编辑模板元数据 (name、type、collegeId、enabled)。不触发版本变更
DELETE	/api/templates/{id}	Admin	删除模板。检查 thesis 表引用, 有关联进行中论文则阻止删除
GET	/api/templates/{id}/versions	Admin	获取指定模板的全部版本列表, 标注当前生效版本
POST	/api/templates/{id}/versions	Admin	创建新版本。系统自动递增版本号, 复制当前版本配置为初始值
PUT	/api/templates/{id}/versions/{vid}/activate	Admin	激活指定版本 (回滚)

coverFields ——封面元素配置 (JSON Array) 封面由多个元素构成, 每个元素的 schema 定义如下:

```

1  [
2    {
3      "field": "title",           // 元素标识
4      "label": "论文题目",       // 封面显示文本
5      "x": 50, "y": 30,         // 位置 (页面百分比坐标)
6      "fontSize": 22,
7      "bold": true
8    },
9    {
10     "field": "date",
11     "label": "提交日期",
12     "x": 50, "y": 90,
13     "fontSize": 14,
14     "format": "yyyy年MM月dd日"
15   }
16 ]

```

chapterStructure ——章节结构配置 (JSON Array) 定义论文的章节树骨架, 递归嵌套结构:

```

1  [
2    {
3      "key": "abstract",         // 章节唯一标识 (对应 ThesisSection.sectionKey)
4      "title": "摘要",          // 章节标题
5      "type": "chapter",        // chapter / section / auto (自动生成)

```

```
6     "children": [                // 子章节数组（递归同结构）
7         { "key": "abstract_cn", "title": "中文摘要", "type": "section" },
8         { "key": "abstract_en", "title": "英文摘要", "type": "section" }
9     ],
10 },
11 { "key": "toc", "title": "目录", "type": "auto" },
12 { "key": "ch1", "title": "引言", "type": "chapter", "children": [...] }
13 ]
```

formatConfig —— 排版格式配置（JSON Object）

```
1 {
2     "page": {
3         "size": "A4",
4         "marginTop": 2.5, "marginBottom": 2.5,
5         "marginLeft": 3.0, "marginRight": 2.5
6     },
7     "font": {
8         "body": "宋体", "bodySize": 12,
9         "heading": "黑体", "headingSize": 16
10    },
11    "spacing": { "lineHeight": 1.5, "paragraphGap": 0.5 },
12    "headerFooter": {
13        "oddHeader": "章标题",
14        "evenHeader": "论文题目",
15        "pageNumber": "第{page}页"
16    }
17 }
```

8 论文模块概要设计

8.1 模块定位

论文模块是系统的业务核心层，管理论文从创建到生成的全生命周期。该模块与章节存取模块紧密协作：论文模块管理论文的元数据和生命周期，章节存取模块管理论文的内容数据。

8.2 论文状态机设计

8.3 论文锁定机制

- 论文提交时 `is_locked` 自动置为 `true`，前端编辑器进入只读模式。
- 教师退回时 `is_locked` 自动置为 `false`，恢复编辑权限。
- 文档生成中 `is_locked` 为 `true`，前端禁用所有编辑操作。
- 仅在 `DRAFT` 状态且 `is_locked` 为 `false` 时，才允许调用章节内容更新接口。

表 11: 论文状态转换规则

当前状态	可转换目标状态	触发条件
DRAFT	SUBMITTED	学生点击提交按钮, 论文锁定
DRAFT	GENERATING	学生点击生成文档按钮
SUBMITTED	REVIEWED	教师提交批阅意见
SUBMITTED	RETURNED	教师点击退回修改
RETURNED	DRAFT	学生进入编辑 (自动切换, 解锁论文)
GENERATING	COMPLETED	文档导出任务执行完成

9 论文模块接口设计

表 12: 论文管理 RESTful API

方法	URL	权限	说明
GET	/api/theses	Student	获取当前学生全部论文列表, 按 updated_at 降序
POST	/api/theses	Student	创建新论文。请求体: { "title": "...", "templateVersionId": 3, "collegeId": 1 }。自动创建章节记录
GET	/api/theses/{id}	Student	获取论文详情 (含章节树)
PUT	/api/theses/{id}	Student	修改论文标题 (仅 DRAFT 状态)
DELETE	/api/theses/{id}	Student	删除论文 (仅 DRAFT 状态), 级联删除章节
PUT	/api/theses/{id}/submit	Student	提交论文: 校验章节非空, 改状态为 SUBMITTED, 锁定

10 章节存取模块概要设计

10.1 章节树存储模型

ThesisSection表采用邻接表模型 (Adjacency List) 存储树形数据:

- 根节点: parent_id 为 NULL, section_type 为"chapter"。
- 子节点: parent_id 指向父章节 id, section_type 为"section", 可继续嵌套。
- 排序: 同层级节点按 sort_order 升序排列。

- **双内容字段：**content存储正式内容，draftContent 存储自动保存草稿。编辑器加载时优先展示 draftContent 以恢复未保存内容。

11 章节存取接口设计

表 13: 章节管理 RESTful API

方法	URL	权限	说明
GET	/api/theses/{id}/sections	Student	获取论文全部章节树（递归嵌套 JSON，按 sort_order 排序）
PUT	/api/sections/{id}	Student	更新章节内容。请求体：{ "content": "..."} 或 { "draftContent": "..."} }
PUT	/api/sections/{id}/submit	Student	提交章节：draftContent 合并到 content，清空 draftContent
POST	/api/sections/{id}/auto-save	Student	自动保存草稿（仅更新 draftContent），每 30s 前端触发
PUT	/api/sections/{id}/reorder	Student	调整章节排序
POST	/api/theses/{id}/sections	Student	新增自定义章节
DELETE	/api/sections/{id}	Student	删除章节（级联删除子章节）

12 数据库表结构汇总

13 参考文献管理模块概要设计

13.1 模块职责

参考文献管理模块负责学生论文写作过程中参考文献条目的全生命周期管理，包括文献信息的 CRUD 操作、分类检索和标准格式化输出。该模块独立于论文实体运行，维护一个通用的参考文献库，导出时按序注入文档。

13.2 模块接口

13.3 与外部模块的交互

- **与论文管理模块：**论文的参考文献数据通过 thesis_id 外键关联，导出模块读取格式化后的参考文献列表写入文档；
- **与前端交互：**前端通过 REST API 完成文献的增删改查操作，格式化结果在前端预览列表展示；

表 14: B 负责模块涉及的数据库表

表名	对应实体	主键	核心字段
sys_college	College	id BIGINT 自增	name, code (UNIQUE), created_at, updated_at
template	Template	id BIGINT 自增	name, type, college_id FK, enabled, created_at, updated_at
template_version	TemplateVersion	id BIGINT 自增	template_id FK, version_number, is_current, cover_fields TEXT(JSON), chapter_structure TEXT(JSON), format_config TEXT(JSON), created_at
thesis	Thesis	id BIGINT 自增	student_id FK, template_version_id FK, title, status, is_locked, college_id FK, created_at, updated_at
thesis_section	ThesisSection	id BIGINT 自增	thesis_id FK, title, section_key, content TEXT, draft_content TEXT, section_type, parent_id 自引用 FK, sort_order, created_at, updated_at

表 15: 参考文献管理模块 API 接口

方法	路径	说明
GET	/api/references	获取所有文献（按年份降序）
GET	/api/references/{id}	获取单条文献
POST	/api/references	新增文献
PUT	/api/references/{id}	更新文献
DELETE	/api/references/{id}	删除文献
GET	/api/references/search/author	按作者搜索
GET	/api/references/search/title	按标题搜索
GET	/api/references/{id}/format	单条 GB/T 7714 格式化
GET	/api/references/format/all	全部 GB/T 7714 格式化列表

- 与导出模块：导出模块调用格式化服务获取 GB/T 7714 格式化的文献字符串列表，按序写入 DOCX 文档的”参考文献”章节。

13.4 GB/T 7714 格式化设计

系统支持五种文献类型的标准化格式化输出：

- [J] 期刊文章：作者. 标题 [J]. 刊物名称, 年份, 卷 (期): 页码.
- [M] 专著/书籍：作者. 标题 [M]. 出版地: 出版社, 年份.
- [C] 会议论文：作者. 标题 [C]// 会议名称. 年份: 页码.
- [D] 学位论文：作者. 标题 [D]. 授予单位, 年份.
- [EB] 网络资源：作者. 标题 [EB/OL]. 链接, 引用日期.

格式化器采用策略模式实现，根据文献类型自动路由到对应的格式化方法，便于后续扩展新的文献类型。

14 图片上传与管理模块概要设计

14.1 模块职责

图片管理模块负责学生论文插图的文件上传、校验、存储和访问。文件存储于服务器本地文件系统，图片元信息存储于数据库。

14.2 模块接口

表 16: 图片管理模块 API 接口

方法	路径	说明
POST	/api/images/upload	上传图片（multipart/form-data）
GET	/api/images/{id}	获取图片元信息
GET	/api/images/{id}/file	获取图片文件（浏览器直接查看）
DELETE	/api/images/{id}	删除图片

14.3 技术方案

- 上传协议：HTTP multipart/form-data，通过 Spring MVC 的 MultipartFile 接收；
- 文件存储：服务器本地文件系统，存储路径为 {uploadDir}/{uuid}.{ext}；

- **文件名策略:** 采用 UUID v4 生成存储文件名, 避免中文文件名编码问题和重名冲突;
- **格式校验:** 后端校验 Content-Type 头, 仅允许 image/jpeg、image/png、image/gif、image/webp;
- **大小限制:** 通过 `spring.servlet.multipart.max-file-size=10MB` 全局限制;
- **访问方式:** 通过 `/api/images/{id}/file` 静态资源映射返回文件流, Content-Disposition 设置为 inline 以支持浏览器直接展示。

14.4 与外部模块的交互

- **与前端交互:** 前端在上传成功后获取图片 URL, 将其嵌入编辑器 HTML 内容中;
- **与导出模块:** 导出模块解析论文章节 HTML, 提取 `<img>` 标签中的图片 URL, 读取本地图片文件嵌入 DOCX/PDF 文档。

15 文档导出模块概要设计

15.1 模块职责

文档导出模块是本系统的核心模块, 负责将学生在 Web 编辑器中撰写的论文内容, 按模板格式配置生成标准的 Word (DOCX) 和 PDF 文档。

15.2 总体流程

1. 学生点击”导出”按钮, 前端发送导出请求 (指定论文 ID 和导出格式);
2. 后端接收请求, 校验论文状态和内容完整性;
3. 后端从数据库读取论文数据 (各章节内容、参考文献列表、模板格式配置);
4. 获取嵌入的图片文件资源 (从本地文件系统读取);
5. 使用 Apache POI XWPF 按模板格式配置创建 DOCX 文档;
6. 若导出格式为 PDF, 则调用 LibreOffice 无头模式将 DOCX 转换为 PDF;
7. 将生成的文档文件返回给前端 (直接下载或返回文件链接)。

15.3 技术方案对比

技术选型结论:

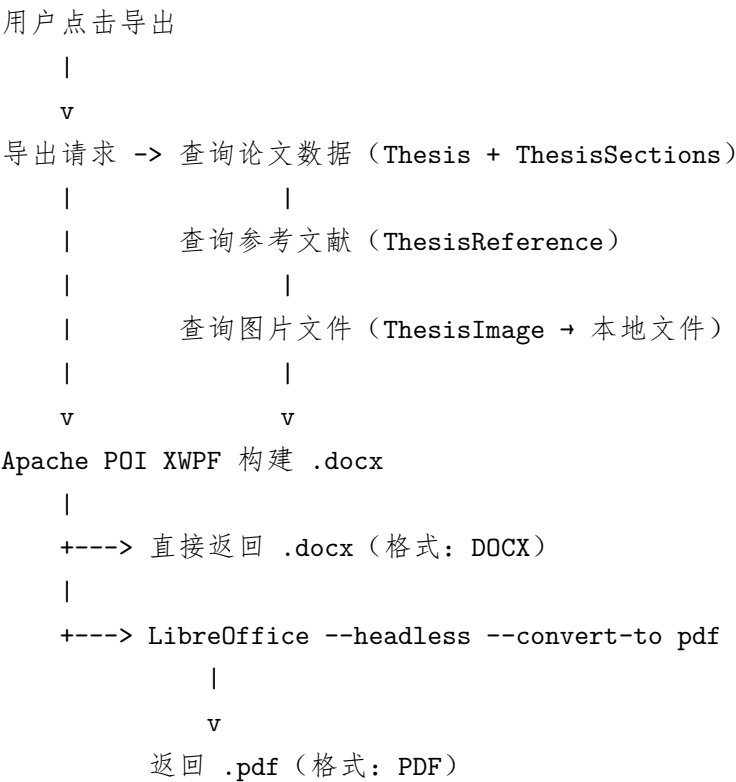
- DOCX 生成采用 Apache POI (poi-ooxml 5.x), 直接操作 XWPFDocument 对象, 逐段落、表格、图片构建文档;

表 17: 文档生成技术方案对比

方案	DOCX 生成	PDF 生成	中文支持
Apache POI	原生创建、样式精细	不支持	需指定中文字体
LibreOffice Headless	—	从 DOCX 转换，还原度高	系统字体依赖

- PDF 生成采用”DOCX → LibreOffice Headless → PDF”的两步方案，保证 DOCX 和 PDF 输出的一致性。

15.4 数据流设计



15.5 与外部模块的交互

- 与论文管理模块：读取论文的章节内容（HTML 格式）和模板格式参数（JSON）；
- 与参考文献模块：调用格式化服务，获取 GB/T 7714 格式化的参考文献字符串列表；
- 与图片管理模块：通过图片存储路径读取本地图片文件，嵌入文档对应位置；
- 与前端交互：通过异步任务接口反馈导出进度，最终返回下载链接。

16 前端技术选型与架构概述

16.1 技术栈

前端采用 Vue 3 生态体系构建单页面应用（SPA），具体选型如下：

表 18: 前端技术栈选型表

类别	技术	版本
框架	Vue 3 (Composition API)	3.4+
构建工具	Vite	5.0+
语言	TypeScript	5.3+
UI 组件库	Element Plus	2.5+
路由	Vue Router	4.3+
HTTP 客户端	Axios	1.6+
状态管理	Pinia	2.1+
富文本编辑器	Tiptap (基于 ProseMirror)	2.0+
代码规范	ESLint + Prettier	8.0+ / 3.0+

16.2 选型理由

- **Vue 3 + Composition API:** 组合式 API 提供更好的逻辑复用能力和 TypeScript 支持；`<script setup>` 语法糖减少样板代码，提高开发效率。
- **Vite:** 基于 ESM 的开发服务器，冷启动速度远快于 Webpack；HMR 毫秒级响应，提升开发体验；生产构建基于 Rollup，产物体积小。
- **Element Plus:** Vue 3 生态最成熟的桌面端 UI 库，提供表格、表单、对话框、树形控件等全套组件，适合论文编辑器的复杂交互场景。
- **Tiptap:** 基于 ProseMirror 的现代化富文本编辑器框架，支持自定义扩展（参考文献引用、图片上传、表格编辑），完全可控的 Schema 设计，渲染与数据分离，便于导出 DOCX/PDF。
- **Pinia:** Vue 官方推荐的状态管理库，TypeScript 原生支持，模块化 Store 设计，比 Vuex 更简洁。

17 前端工程目录结构

```
thesis-generator-web/  
  public/                # 静态资源  
    favicon.ico  
  src/  
    api/                 # API接口封装层  
      request.ts         # Axios实例 + 请求/响应拦截器
```

```

7      auth.ts          # 认证相关API
8      paper.ts         # 论文CRUD API
9      section.ts       # 章节API
10     template.ts      # 模板API
11     reference.ts      # 参考文献API
12     export.ts        # 导出API
13     college.ts       # 学院API
14 assets/              # 样式/图片/字体
15     styles/
16         variables.scss # SCSS变量（主题色、间距等）
17         global.scss    # 全局样式
18         editor.scss    # 编辑器专属样式
19     images/
20 components/          # 全局共享组件
21     common/           # 通用组件（Header、Loading等）
22     editor/           # 编辑器组件
23         EditorToolbar.vue # 编辑器工具栏
24         SectionTree.vue  # 章节树组件
25         ReferencePanel.vue # 参考文献管理面板
26         ImageUploader.vue # 图片上传组件
27         TableEditor.vue  # 表格编辑组件
28     preview/
29         PaperPreview.vue
30 composables/         # 组合式函数（逻辑复用）
31     useAuth.ts        # 认证逻辑
32     useAutoSave.ts    # 自动保存逻辑
33     useEditor.ts      # 编辑器核心逻辑
34     useExport.ts      # 导出逻辑
35 layouts/             # 布局组件
36     DefaultLayout.vue # 默认布局（Header+内容+Footer）
37     EditorLayout.vue  # 编辑器三栏布局
38     AuthLayout.vue   # 认证页居中布局
39 router/              # 路由配置
40     index.ts
41 stores/              # Pinia状态管理
42     auth.ts          # 用户认证状态
43     paper.ts         # 当前论文状态
44     editor.ts        # 编辑器状态（当前章节、脏数据标记等）
45 types/              # TypeScript类型定义
46     paper.ts
47     section.ts
48     template.ts
49     api.ts
50 utils/              # 工具函数
51     token.ts         # Token存取
52     validator.ts     # 表单校验规则
53 views/              # 页面视图组件
54     auth/

```

```
55     Login.vue
56     Register.vue
57   paper/
58     PaperList.vue      # 论文列表首页
59     PaperCreate.vue    # 论文创建向导
60     PaperEditor.vue    # 论文编辑器主页面
61   preview/
62     PreviewPage.vue    # 预览页面
63   user/
64     Profile.vue        # 个人中心
65     ExportHistory.vue  # 导出历史
66   App.vue
67   main.ts              # 入口文件
68   vite.config.ts
69   tsconfig.json
70   package.json
71   .env.development / .env.production
```

18 前端路由设计

18.1 路由表定义

系统采用 Vue Router 4 的嵌套路由方案，结合路由元信息（meta）实现鉴权控制和布局切换。

表 19: 前端路由表

路径	页面组件	布局	鉴权
/login	Login	AuthLayout	否
/register	Register	AuthLayout	否
/papers	PaperList	DefaultLayout	是
/papers/create	PaperCreate	DefaultLayout	是
/editor/:paperId	PaperEditor	EditorLayout	是
/preview/:paperId	PreviewPage	DefaultLayout	是
/profile	Profile	DefaultLayout	是
/export-history	ExportHistory	DefaultLayout	是

18.2 路由鉴权守卫

```
1 // router/index.ts
2 router.beforeEach((to, from, next) => {
3   const token = localStorage.getItem('token')
4   if (to.meta.requiresAuth && !token) {
5     next({ path: '/login', query: { redirect: to.fullPath } })
6   } else if (token && (to.path === '/login' || to.path === '/register')) {
```

```
7     next({ path: '/papers' })
8   } else {
9     next()
10  }
11 })
```

所有页面组件采用动态 `import()` 实现路由级代码分割，减少首屏加载体积。

19 前端组件架构设计

19.1 组件分层架构

前端组件按职责划分为四层：

1. **页面层 (Views)**: 路由级页面组件，负责页面整体布局编排、调用 API 获取数据、协调子组件交互。不包含具体的 UI 渲染逻辑。
2. **业务组件层 (Business Components)**: 承载特定业务逻辑的复合组件，如章节树 (Section-Tree)、参考文献面板 (ReferencePanel)、导出对话框 (ExportDialog)。这些组件内部管理自身状态，通过 Props 接收数据、Emits 向外传递事件。
3. **通用组件层 (Common Components)**: 无业务耦合的纯 UI 组件，如全局 Header、Loading 遮罩、确认对话框。可在多个页面复用。
4. **基础组件层 (UI Library)**: 直接使用 Element Plus 提供的按钮、输入框、表格、对话框、树形控件等原子级组件。

19.2 核心组件通信模式

- **父子通信**: 父组件通过 Props 传递数据给子组件，子组件通过 `defineEmits` 向父组件发送事件通知。适用于章节树 → 编辑器的章节切换、工具栏 → 编辑器的格式操作。
- **全局状态 (Pinia Store)**: 跨页面共享的数据使用 Pinia 管理。主要包括:
 - `authStore`: 当前登录用户信息、Token、登录/登出操作；
 - `paperStore`: 当前论文基本信息、章节列表、模板信息；
 - `editorStore`: 当前编辑章节 ID、脏数据标记、自动保存状态、编辑器实例引用。
- **依赖注入 (provide/inject)**: 编辑器内部深层嵌套的组件通过 `provide/inject` 获取编辑器实例和上下文配置，避免 Prop 逐层传递。

19.3 编辑器组件架构

编辑器是学生端最复杂的业务组件，采用 Tiptap + 自定义扩展的架构：

1. **Tiptap Editor 实例**：通过 `useEditor()` 组合式函数创建，配置插件扩展集（Bold、Italic、Heading、Image、Table、ReferenceCitation 等）。
2. **自定义 Extension**：开发 ReferenceCitation 扩展节点，在编辑器文档中以 inline 节点存储引用 ID，渲染为 [n] 标记，导出时转换为格式化引用。
3. **工具栏 (EditorToolbar)**：通过 Tiptap 的 `editor.chain().focus()` API 调用编辑器命令，每个按钮绑定对应命令，高亮当前激活状态。
4. **章节树 (SectionTree)**：使用 Element Plus 的 Tree 组件，支持拖拽排序（node-drop 事件），右键上下文菜单（添加/删除/重命名章节）。
5. **自动保存 (useAutoSave)**：监听编辑器 `onUpdate` 事件，使用防抖（debounce）机制，30 秒无新修改后自动触发保存 API；页面关闭前通过 `beforeunload` 事件检测脏数据。
6. **参考文献面板 (ReferencePanel)**：侧边栏面板，使用 Element Plus Table 展示文献列表，Dialog 承载添加/编辑表单。与编辑器通过 Pinia Store 通信，正文引用标记与文献列表保持同步。

19.4 组件状态生命周期

以论文编辑器页面为例，组件状态流转如下：

1. 页面挂载阶段：

- 从路由参数获取 `paperId`；
- 调用 `GET /api/v1/papers/:paperId` 获取论文基础信息，存入 `paperStore`；
- 调用 `GET /api/v1/papers/:paperId/sections` 获取章节树，存入 `editorStore`；
- 初始化 Tiptap 编辑器实例，加载第一个章节内容；
- 启动自动保存监听。

2. 编辑交互阶段：

- 切换章节 → 保存当前章节 → 加载目标章节内容至编辑器；
- 拖拽排序 → 更新本地章节顺序 → 发送 PUT 请求更新后端；
- 添加章节 → 弹出命名对话框 → 确认后创建章节节点 → 刷新章节树；
- 插入引用 → 打开参考文献面板 → 选择文献 → 光标处插入引用标记。

3. 页面卸载阶段：

- 检查脏数据标记，若有未保存修改则弹窗提示；
- 执行最终保存；
- 销毁 Tiptap 编辑器实例，释放内存；
- 清理 Pinia Store 中的临时状态。

20 Axios 封装与 API 层设计

20.1 请求/响应拦截器

```
1 // api/request.ts
2 import axios from 'axios'
3 import { ElMessage } from 'element-plus'
4 import router from '@/router'
5
6 const request = axios.create({
7   baseURL: import.meta.env.VITE_API_BASE_URL || '/api/v1',
8   timeout: 15000,
9 })
10
11 request.interceptors.request.use((config) => {
12   const token = localStorage.getItem('token')
13   if (token) {
14     config.headers.Authorization = `Bearer ${token}`
15   }
16   return config
17 })
18
19 request.interceptors.response.use(
20   (response) => response.data,
21   (error) => {
22     if (error.response?.status === 401) {
23       localStorage.removeItem('token')
24       router.push('/login')
25       ElMessage.error('登录已过期，请重新登录')
26     } else {
27       ElMessage.error(error.response?.data?.message || '请求失败')
28     }
29     return Promise.reject(error)
30   }
31 )
32
33 export default request
```


表 20: 学生端调用 API 接口汇总

方法	路径	说明
POST	/api/v1/auth/register	学生注册
POST	/api/v1/auth/login	学生登录
POST	/api/v1/auth/logout	退出登录
GET	/api/v1/papers	获取论文列表
POST	/api/v1/papers	新建论文
GET	/api/v1/papers/:id	获取论文详情
PUT	/api/v1/papers/:id	更新论文信息
DELETE	/api/v1/papers/:id	删除论文
GET	/api/v1/papers/:id/sections	获取章节树
POST	/api/v1/papers/:id/sections	新增章节
GET	/api/v1/papers/:id/sections/:sid	获取章节内容
PUT	/api/v1/papers/:id/sections/:sid	保存章节内容
PUT	/api/v1/papers/:id/sections/order	更新章节排序
DELETE	/api/v1/papers/:id/sections/:sid	删除章节
GET	/api/v1/templates	获取模板列表
GET	/api/v1/templates/:id	获取模板详情
GET	/api/v1/references	获取参考文献列表
POST	/api/v1/references	新增参考文献
PUT	/api/v1/references/:id	编辑参考文献
DELETE	/api/v1/references/:id	删除参考文献
POST	/api/v1/papers/:id/export	提交导出任务
GET	/api/v1/export/:taskId/status	查询导出进度
GET	/api/v1/export/history	获取导出历史
POST	/api/v1/images/upload	上传图片

20.2 学生端 API 接口清单

21 学生端核心模块概要设计

21.1 论文创建向导流程

1. 用户在论文列表页点击”新建论文”，前端弹出 PaperCreate 对话框或跳转独立页面；
2. 步骤一：输入论文标题，选择所属学院。学院列表通过 API 获取并在下拉框中展示；
3. 步骤二：根据所选学院 ID 获取匹配模板列表。前端以卡片形式展示模板名称、封面缩略图、适用说明；
4. 步骤三：用户确认创建，前端组装论文信息（标题、学院 ID、模板 ID）发送 POST 请求；
5. 后端返回新论文 ID，前端跳转至编辑器页面 /editor/:paperId。

21.2 编辑器核心逻辑

1. 页面加载：获取论文数据 → 获取章节树 → 初始化编辑器 → 加载默认章节内容 → 启动自动保存；
2. 章节切换：保存当前章节（若有修改）→ 发送 PUT 请求 → 获取新章节内容 GET → 重置编辑器内容；
3. 章节拖拽排序：监听 node-drop 事件 → 本地更新顺序 → 发送 PUT 请求提交新顺序；
4. 自动保存：编辑器 onUpdate 事件 → 防抖 30 秒 → 标记脏数据 → 发送 PUT 请求 → 更新最后保存时间显示；
5. 图片上传：用户粘贴或拖拽图片 → 前端压缩（可选）→ POST 上传 → 返回 URL → 插入编辑器。

21.3 论文预览

1. 用户点击”预览”按钮，前端调用 API 获取预览数据；
2. 后端根据论文模板样式，将章节内容渲染为 HTML；
3. 前端在新标签页展示渲染后的 HTML，页面布局模拟 A4 纸张效果；
4. 预览页支持翻页浏览、缩放查看功能。

22 管理员端页面概要设计

22.1 页面布局

管理员端采用侧边导航 + 顶部栏 + 内容区域的经典后台管理布局。侧边导航收起时仅显示图标，展开时显示图标 + 文字标签。顶部栏展示系统 Logo、当前管理员用户名、退出登录按钮。内容区域根据路由切换展示不同的功能页面。

管理员端包含以下核心页面：

- 仪表盘页（`/admin/dashboard`）：系统数据概览看板，展示用户数、论文数、模板数等核心指标；
- 学院管理页（`/admin/colleges`）：学院信息的 CRUD 操作界面；
- 用户管理页（`/admin/users`）：用户账号的查询、筛选、禁用/启用操作界面；
- 模板管理页（`/admin/templates`）：模板列表展示、创建、编辑、启用/停用操作界面；
- 模板版本编辑页（`/admin/templates/:id/versions`）：JSON 配置编辑器，管理模板的封面字段、章节结构和格式参数。

22.2 仪表盘页设计

仪表盘页采用卡片网格布局，上方为四个统计指标卡片（总用户数、在编论文数、模板总数、今日导出数），下方为图表区域（论文状态分布饼图、每天导出数量趋势折线图）。所有图表数据通过独立 API 获取，支持按时间范围筛选。

22.3 学院管理页设计

学院管理页采用表格视图，上方为操作栏（新增学院按钮、搜索输入框），主体为 `el-table` 组件展示学院列表，字段包含学院名称、学院代码、关联模板数、关联用户数、创建时间。表格行末尾提供编辑和删除操作按钮。新增/编辑操作通过 `el-dialog` 弹窗实现。

22.4 模板管理页设计

模板管理页是管理员端配置最复杂的页面，采用左侧列表 + 右侧编辑的双栏布局：

- 左侧模板列表以卡片形式展示模板名称、类型标签（毕业/课程/项目）、当前版本号、启用状态开关（`el-switch`），支持按类型和学院筛选；
- 右侧编辑区域在未选中模板时展示“请选择模板”占位提示，选中模板后切换为编辑界面；
- 编辑界面采用 `Tabs` 组件组织三个配置面板：封面字段配置（拖拽排序的字段列表）、章节结构配置（树形编辑器）、格式参数配置（表单）。

22.5 模板版本编辑页设计

版本编辑页右侧使用 JSON 编辑器或可视化表单进行配置编辑。封面字段配置区支持拖拽添加/删除/排序封面元素，每个元素可配置字段标签、是否必填、默认值。章节结构采用 el-tree 组件，支持拖拽调整层级、右键菜单新增/删除节点。格式参数采用 el-form 配合 el-color-picker（颜色选择）、el-slider（行距/边距调整）等丰富控件，参数调整后右侧实时预览展示渲染效果。

23 教师端页面概要设计

23.1 页面布局

教师端同样采用侧边导航 + 顶部栏 + 内容区域布局。侧边导航包含批阅论文列表和批阅历史两个主要入口。教师端整体风格偏阅读和审阅，内容区域以白底卡片为主，减少视觉干扰。

教师端包含以下核心页面：

- 批阅任务列表页（/teacher/reviews）：展示待批阅和已批阅的论文列表；
- 论文批阅页（/teacher/reviews/:id）：论文全文阅读 + 批注添加 + 评分的核心操作页面；
- 批阅历史页（/teacher/history）：展示教师历次批阅记录汇总。

23.2 批阅任务列表页设计

批阅任务列表采用表格视图，展示字段包括论文标题、学生姓名、所属学院、提交时间、当前状态（待批阅/已批阅/已退回）。表格支持按学院筛选和按提交时间排序。状态列使用不同颜色的 el-tag 展示：待批阅为橙色、已批阅为绿色、已退回为灰色。新增提交的论文在列表顶部显示，并用闪烁的“新”标签提示教师。

23.3 论文批阅页设计

论文批阅页采用三栏布局，是本系统交互最复杂的页面之一：

1. 左侧栏（章节导航，宽度 240px）：使用 el-tree 展示论文的完整章节树，当前批阅章节高亮显示。教师点击章节名称切换预览内容。章节标题旁标注该章节的批注数量；
2. 中间栏（论文预览，自适应宽度）：以 HTML 渲染方式展示论文章节内容，模拟 A4 纸张样式。页面支持滚动浏览。教师拖拽选中文字后，选中区域以主题色高亮显示，并弹出浮动工具栏（添加批注按钮）。已添加批注的文字以黄色下划线标记，悬停时显示批注预览；
3. 右侧栏（批注面板，宽度 320px）：分为上下两部分。上部为批注列表区，展示当前章节的全部批注（按创建时间排序），每条批注显示批注内容摘要和创建时间。下部为批注编辑区，当教师选中文字后自动填充 selected_text 预览，并提供 textarea 输入批注内容。

23.4 评分对话框设计

教师完成批注后，点击页面顶部的”评阅完成”按钮触发评分对话框。对话框采用 el-dialog 组件，宽度 600px，包含以下控件：

- 评语编辑器（el-input type=”textarea”，支持富文本格式，最大 5000 字符）；
- 评分滑块（el-slider，范围 0-100，步进 1，显示当前分值大号数字）；
- 等级展示（根据分数自动换算并显示，如”85 分 → 良”，带颜色标签）；
- 操作类型单选组（el-radio-group）：通过（REVIEWED）/ 退回（RETURNED）；
- 退回原因输入框（条件显示，仅当选择”退回”时出现，el-input，最小 10 字符）；
- 确认提交按钮和取消按钮。

24 UI 设计规范

24.1 颜色系统

为保证系统视觉一致性，定义以下颜色令牌：

表 21：UI 颜色系统规范

用途	色值	Element Plus 变量	适用范围
主色	#409EFF	-el-color-primary	按钮、链接、导航选中态
成功	#67C23A	-el-color-success	成功提示、已完成标签
警告	#E6A23C	-el-color-warning	警告提示、待处理标签
危险	#F56C6C	-el-color-danger	错误提示、删除操作、已退回标签
信息	#909399	-el-color-info	辅助文字、禁用状态
背景	#F5F7FA	—	页面背景色、卡片背景色

24.2 字号层级

表 22：字号层级规范

层级	字号	应用场景
页面标题	20px / 18pt	各功能页面的主标题
卡片标题	16px / 14pt	卡片组件标题、Dialog 标题
正文/表头	14px / 12pt	表格内容、表单标签、段落文字
辅助文字	12px / 10.5pt	提示信息、时间戳、标签文字

24.3 间距规范

- 页面内边距：内容区域与页面边缘保持 24px 间距；
- 卡片内边距：卡片容器内边距 20px；
- 组件间距：相邻表单控件之间间距 16px；
- 标签与控件间距：表单标签与输入框之间间距 12px；
- 段落间距：卡片内多段文字间距 8px。

24.4 响应式断点

系统面向桌面端使用，但要求在不同屏幕尺寸下保持可用：

- 大屏（ $\geq 1200\text{px}$ ）：三栏布局完整展开，适合在 1920×1080 显示器上使用；
- 中屏（768-1199px）：右侧面板自动收起为浮动按钮，点击后以 Drawer 形式展开；
- 小屏（ $< 768\text{px}$ ）：切换为单栏布局，侧边导航自动收起为汉堡菜单。

24.5 组件使用约定

- 表格（el-table）：数据量超过 20 条时分页，每页 20 条。表格行支持 hover 高亮，关键操作的按钮在操作列展示；
- 表单（el-form）：必填字段标红色星号，提交前做前端校验，校验不通过时在字段下方显示错误提示文字；
- 对话框（el-dialog）：宽度统一为 600px 或 800px，关闭方式支持点击遮罩关闭和右上角 X 关闭。提交操作后自动关闭并刷新父页面数据；
- 消息提示（ElMessage）：操作成功使用绿色提示（成功），操作失败使用红色提示（错误），持续时间均为 3 秒；
- 加载状态（v-loading）：数据获取和表单提交过程中显示加载遮罩，防止重复操作。

25 缓存与部署架构设计

本章描述系统在缓存层与部署层的概要设计。缓存层基于 Redis 构建，为高频读取数据、用户会话及异步任务提供内存级访问加速；部署层采用 Docker 容器化方案，实现应用服务、数据库与缓存的统一编排与一键部署。

25.1 缓存架构概述

ThesisGenerator 系统的读操作远多于写操作——学院信息、模板结构、参考文献格式等数据变更频率极低但查询频繁；论文章节在编辑过程中需要高频自动保存；JWT 令牌需要快速校验与失效管理。引入 Redis 作为统一缓存层，可显著降低 PostgreSQL 的读负载，提升系统整体响应速度。

25.1.1 缓存架构分层

系统缓存架构分为三层，自上而下依次为：

1. **浏览器缓存层**：前端 SPA 的静态资源（JS/CSS/字体）通过 HTTP Cache-Control 头缓存于浏览器，Template 结构等不常变数据缓存在 Vuex/Pinia 状态管理中，减少重复 API 请求。
2. **Redis 缓存层（核心）**：部署于应用服务器与数据库之间，承载四项职责：
 - 业务数据缓存：学院列表、模板版本、参考文献格式等读多写少数据。
 - 会话与令牌缓存：JWT Token 黑名单、Refresh Token、用户会话状态。
 - 草稿暂存：学生编辑过程中的章节草稿定时自动保存于 Redis，实现跨设备的编辑进度同步。
 - 限流与锁：登录接口的令牌桶限流计数器、导出任务的分布式锁。
3. **数据库持久层**：PostgreSQL 作为最终数据源，Redis 中所有缓存数据均有对应的数据库表，缓存失效后可回源查询重建。

25.1.2 缓存数据流

典型的数据读取流程如下：

1. 客户端发起 API 请求 → Controller 层接收。
2. Service 层先查询 Redis 缓存，若命中（Cache Hit）则直接返回，耗时 $\leq 5\text{ms}$ 。
3. 若未命中（Cache Miss），Service 查询 PostgreSQL，将结果序列化写入 Redis（设置合理 TTL），再返回给客户端。
4. 当源数据发生变更（增/删/改），Service 在事务提交后主动删除或更新对应的 Redis 缓存项（Cache-Aside 模式）。

25.2 Redis 数据结构设计

根据缓存数据的访问模式与生命周期，选用以下 Redis 数据结构：

表 23: Redis 缓存项设计一览

Key 模式	用途	类型	TTL	更新策略
token:<userId>	JWT 令牌存储	String	24h	登录写入/登出删除
refresh:<userId>	Refresh Token	String	7d	登录写入/轮换更新
blacklist:<tokenId>	Token 黑名单	String	24h	登出/改密写入
college:list	学院全列表	String(JSON)	1h	学院变更时删除
template:<id>	模板详情	Hash	30min	模板更新时删除
template:version:<id>	模板版本	Hash	30min	版本变更时删除
draft:<thesisId>:<sectionId>	章节草稿	String(JSON)	30min	定时自动保存覆盖
rate:<ip>:<endpoint>	限流计数器	String	1min	每次请求递增
lock:export:<thesisId>	导出分布式锁	String	30s	导出开始设/结束删

25.2.1 序列化方案

采用 `GenericJackson2JsonRedisSerializer` 作为默认值序列化器，相较于 `JDK` 序列化具有以下优势：

- 可读性好：缓存内容为 `JSON` 格式，便于运维排查。
- 跨语言兼容：`JSON` 为通用格式，未来若引入其他语言微服务可直接读取。
- 无需实现 `Serializable` 接口：对实体类零侵入。

`Key` 统一使用 `StringRedisSerializer`，保证 `Key` 的可读性。

25.2.2 连接池配置

`Redis` 客户端采用 `Lettuce`（`Spring Boot Data Redis` 默认实现），基于 `Netty` 异步 `I/O`，天然支持集群与哨兵模式。连接池参数设计如下：

表 24: Lettuce 连接池配置

参数	值	说明
max-active	50	最大活跃连接数
max-idle	10	最大空闲连接数
min-idle	5	最小空闲连接数
timeout	3000ms	连接超时时间

25.3 缓存策略详解

25.3.1 缓存预热

系统启动时通过 `@PostConstruct` 或 `ApplicationRunner` 将学院列表、当前生效的模板结构等启动必需数据预加载至 `Redis`，避免首批用户请求的冷启动延迟。

25.3.2 缓存失效策略

- **主动失效(Cache-Aside):**数据更新时,Service 层在数据库事务提交后显式调用 `redisTemplate.delete(key)` 删除对应缓存。下次读取时自动触发缓存重建。
- **被动失效 (TTL 过期):** 每种缓存项设置合理 TTL, 即使主动失效遗漏, 也能在 TTL 到期后自动清理, 防止脏数据长期驻留。
- **批量失效:** 模板结构变更可能影响多个版本的缓存。采用 Key 前缀扫描 (`template:*`) 批量删除关联缓存项 (生产环境慎用 `KEYS` 命令, 改用 `SCAN` 游标迭代)。

25.3.3 缓存穿透防护

对于查询不存在的数据 (如查询不存在的论文 ID), 若每次都穿透到数据库, 可能被恶意利用造成数据库压力。防护措施:

- 对查询结果为空的情况, 缓存一个空值标记 (如"NULL"), TTL 设为较短的 60 秒。
- 在 Service 层对查询参数进行合法性校验 (如 ID 必须为正整数), 非法参数直接返回。

25.3.4 缓存雪崩防护

大量缓存项在同一时刻过期, 导致瞬时流量全部打到数据库。防护措施:

- 为不同缓存项的 TTL 添加随机偏移量 ($\pm 10\%$), 避免集中过期。
- 核心缓存项 (如学院列表) 设置不过期或超长 TTL, 仅在数据变更时主动刷新。

25.3.5 缓存击穿防护

热点数据 (如当前学期模板) 在缓存过期瞬间被大量并发请求击穿到数据库。防护措施:

- 使用 Redis `SETNX` 命令实现简单的分布式互斥锁, 仅允许第一个未命中缓存的线程去查询数据库并重建缓存, 其余线程等待或返回旧值。

25.4 部署架构概述

ThesisGenerator采用单机容器化部署方案 (支持向集群平滑扩展), 以 Docker Compose 编排三个核心服务: Spring Boot 应用、PostgreSQL 数据库、Redis 缓存。

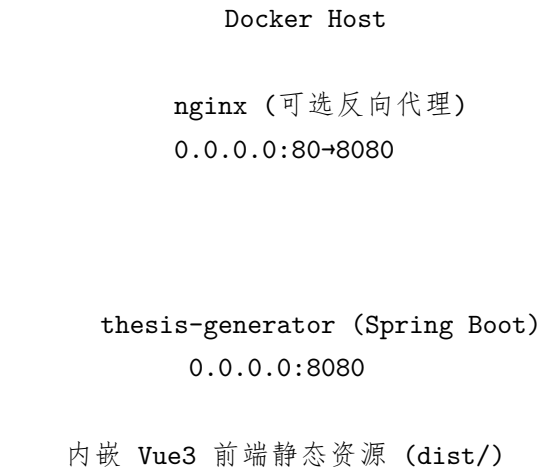


图 2: 系统容器拓扑图

表 25: 部署组件清单

组件	镜像/版本	端口	职责
thesis-generator	自构建 (openjdk:21-jre-slim)	8080	Spring Boot 业务服务
postgres	postgres:16-alpine	5432	关系数据库持久化
redis	redis:7-alpine	6379	缓存与会话存储

表 26: 数据卷与挂载点

数据卷	容器内路径	用途
pgdata	/var/lib/postgresql/data	数据库文件持久化
redisdata	/data	Redis RDB/AOF 持久化
uploads	/app/uploads/images	上传图片存储

25.4.1 容器拓扑

25.4.2 各组件说明

25.4.3 数据卷规划

25.4.4 网络规划

所有容器加入同一 Docker 自定义桥接网络 `thesis-net`，通过容器名进行 DNS 解析（如 PostgreSQL 的 JDBC URL 中 host 写 `postgres` 而非 `localhost`）。不对外暴露 PostgreSQL（5432）与 Redis（6379）端口（仅应用容器内部访问），仅暴露应用端口 8080。

25.4.5 健康检查与重启策略

- **应用健康检查**：通过 Spring Boot Actuator 的 `/actuator/health` 端点，Docker Compose 配置 `healthcheck` 指令定期探测。连续 3 次失败标记为 `unhealthy`。
- **依赖健康检查**：应用容器配置 `depends_on` 与 `condition: service_healthy`，确保 PostgreSQL 与 Redis 先于应用启动并就绪。
- **重启策略**：所有容器配置 `restart: unless-stopped`，保证宿主机重启后服务自动恢复。

25.4.6 环境配置分层

采用 Spring Profile 实现多环境隔离：

表 27：多环境配置策略

Profile	数据库	Redis	JPA DDL
dev	localhost:5432	localhost:6379	update
test	test-db:5432	test-redis:6379	update
prod	prod-db:5432	prod-redis:6379	validate

通过 Docker Compose 的 `environment` 字段传入 `SPRING_PROFILES_ACTIVE=prod`，覆盖 `application.properties` 中的默认值。

25.4.7 资源限制

为每个容器设置 CPU 与内存上限，防止单个服务异常时耗尽宿主机资源：

- **应用容器**：CPU 上限 2 核，内存上限 1GB。
- **PostgreSQL**：CPU 上限 1 核，内存上限 512MB。
- **Redis**：CPU 上限 0.5 核，内存上限 256MB。

25.4.8 部署流程

1. 构建应用 JAR: `./mvnw clean package -DskipTests`。
2. 构建 Docker 镜像: `docker build -t thesis-generator:latest .`。
3. 一键启动: `docker-compose up -d` (自动拉取 postgres 与 redis 镜像, 启动全部服务)。
4. 验证: 访问 `http://localhost:8080/actuator/health` 确认所有组件就绪。
5. 停止: `docker-compose down` (保留数据卷); `docker-compose down -v` (销毁数据卷)。